



Лабораторная работа № 1

От условия до проверенного решения: энергоаудит поездки

Курс	1 курс, 1 семестр	Формат	выполнение + защита
Объём	4 академических часа	Максимум	10 баллов

Работа проверяет полный цикл решения: понять модель, выбрать типы, реализовать вычисления, получить воспроизводимую сборку, доказать результат тестами и проверить советы AI.

1. Результаты обучения

После выполнения работы вы сможете:

- подготовить рабочее окружение для программ на С и доказать, что компилятор доступен из терминала;
- выделить входные и выходные данные, ограничения и зависимости между вычислениями;
- реализовать линейный алгоритм на С с корректными типами, преобразованиями и форматированным вводом-выводом;
- собрать программу с предупреждениями компилятора и получить чистую сборку;
- подобрать тесты, заранее вычислить ожидаемые результаты и объяснить расхождения;
- использовать AI как рецензента или генератор идей для тестирования, но проверять каждое принятое предложение.

2. Что нужно сдать

Артефакт	Содержание
Папка проекта	main.c, README.md и папка evidence с подтверждениями запуска/проверки
Работающая программа	Собирается указанной командой без ошибок и предупреждений; результаты соответствуют модели
README.md	Постановка задачи, таблица данных, формулы, псевдокод, инструкция сборки, тесты, AI-протокол, вывод
AI-протокол	Промпт, значимая часть ответа AI и таблица решений: принято / отклонено / изменено + способ проверки
Защита	Объяснение алгоритма, типов, одного теста и одной проверки AI-предложения; небольшое изменение по просьбе преподавателя



Рекомендуемая структура папки

```
lab01_trip_audit/  
|-- main.c  
|-- README.md  
`-- evidence/  
    |-- environment.png  
    |-- build.png  
    `-- run_variant.png
```

Важно. Скриншоты подтверждают выполнение, но не заменяют исходный код, тестовую таблицу и устную защиту.

3. Этап 1. Подготовка рабочего окружения

3.1. Windows: рекомендуемая конфигурация

1. Установите Visual Studio Code и расширение C/C++ от Microsoft. CMake Tools для этой однофайловой работы не требуется.
2. Установите MSYS2. Запустите терминал, например MSYS2 UCRT64.
3. Установите набор GCC/GDB командой ниже (если вы еще не установили) и добавьте C:\msys64\ucrt64\bin в пользовательскую переменную PATH.
4. Если вы установили только GCC, установить GDB можно сделующей командой: `pacman -S mingw-w64-x86_64-gdb`
5. Полностью перезапустите VS Code и терминалы после изменения PATH.

Команда в MSYS2 UCRT64

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

3.2. Проверка установки

Команды в новом терминале VS Code или Windows Terminal

```
gcc --version  
gdb --version  
code --version
```

Сохраните один скриншот `environment.png`, на котором видны команды и первые строки результатов. Версия может отличаться от примера преподавателя.

3.3. Диагностическая программа

Файл `hello.c`

```
#include <stdio.h>  
  
int main(void) {  
    printf("C environment is ready!\n");  
    return 0;  
}
```

Сборка и запуск в Windows

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -g hello.c -o hello.exe  
.\hello.exe
```

Контрольная точка. Если `hello.c` не собирается из терминала, не переходите к основной задаче: сначала исправьте PATH, имя файла или команду сборки.



4. Этап 2. Постановка задачи

Разработайте консольную программу для расчёта показателей поездки электромобиля. Пользователь вводит параметры поездки и тарифа, программа формирует краткий отчёт.

4.1. Входные данные

Имя в модели	Смысл	Единица	Ограничение
distance_km	длина маршрута	км	> 0
duration_minutes	длительность поездки	мин	целое > 0
battery_capacity_kwh	полная ёмкость батареи	кВт*ч	> 0
start_charge_percent	заряд в начале	%	$0 \leq \text{finish} < \text{start} \leq 100$
finish_charge_percent	заряд в конце	%	$0 \leq \text{finish} < \text{start} \leq 100$
tariff_byn_per_kwh	стоимость энергии	BYN/(кВт*ч)	≥ 0
passengers	число пассажиров	чел.	целое > 0

В ЛР № 1 программа работает с данными, удовлетворяющими указанным ограничениям. Проверка некорректного ввода потребует ветвлений. Здесь ограничения должны быть явно записаны как контракт.

4.2. Выходные данные и формулы

Результат	Формула
time_hours	$\text{duration_minutes} / 60.0$
energy_used_kwh	$\text{battery_capacity_kwh} * (\text{start_charge_percent} - \text{finish_charge_percent}) / 100.0$
average_speed_kmh	$\text{distance_km} / \text{time_hours}$
consumption_kwh_per_100km	$\text{energy_used_kwh} / \text{distance_km} * 100.0$
trip_cost_byn	$\text{energy_used_kwh} * \text{tariff_byn_per_kwh}$
cost_per_passenger_byn	$\text{trip_cost_byn} / \text{passengers}$

В отчёте выведите исходные данные и все шесть вычисленных значений. Для основных результатов используйте две цифры после десятичной точки; при отладке разрешается временно выводить больше знаков.

4.3. Обязательный анализ до кода

1. Заполните таблицу входов/выходов в README.md и обоснуйте тип каждой переменной.
2. Запишите формулы в порядке зависимостей: нельзя вычислить скорость до перевода времени в часы.
3. Составьте псевдокод из 8-12 ясных шагов.



4. Вручную рассчитайте ожидаемый результат для демонстрационного набора хотя бы с четырьмя знаками до округления.
5. Назовите минимум три риска: целочисленное деление, неверные скобки, несоответствие спецификатора формата типу.

4.4. Демонстрационный набор

Параметр	Значение
distance_km	120.5
duration_minutes	95
battery_capacity_kwh	64.0
start_charge_percent	82.0
finish_charge_percent	54.5
tariff_byn_per_kwh	0.42
passengers	3

Ожидаемые основные строки отчёта

```
Energy used: 17.60 kWh
Average speed: 76.11 km/h
Consumption: 14.61 kWh/100 km
Trip cost: 7.39 BYN
Cost per passenger: 2.46 BYN
```

5. Этап 3. Реализация

5.1. Технические требования

- ☐ Стандарт C17; исходный файл main.c; точка входа int main(void).
- ☐ Только темы лекций 1-3: последовательность операций, переменные, выражения, scanf(), printf().
- ☐ Для вещественных величин используйте double; для целочисленных - int. Объясните каждый явный cast.
- ☐ Спецификаторы scanf() должны точно соответствовать типам: %lf для double и %d для int.
- ☐ Имена переменных должны отражать смысл и единицы измерения. Не используйте a, b, x для всех величин.
- ☐ В обычной сборке не должно быть предупреждений с обязательными флагами.
- ☐ Запрещены system("pause"), conio.h, глобальные переменные и конструкции.

Обязательная команда сборки и запуска

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -g main.c -o trip_audit.exe
.\trip_audit.exe
```



Не прячьте предупреждения. Флаг, отключающий предупреждение, не считается исправлением. Нужно устранить причину или обосновать безопасное преобразование.

5.2. Индивидуальный контрольный набор

Номер варианта V (от 1 до 30) назначает преподаватель. В README.md внесите свои входы, самостоятельно вычисленные ожидания и фактический вывод программы.

V	км	мин	кВт*ч	start %	finish %	ВУН/кВт*ч	чел.
1	81.5	57	46.0	87.3	42.6	0.295	1
2	88.0	64	48.0	86.6	43.2	0.310	2
3	94.5	71	50.0	85.9	43.8	0.325	3
4	101.0	78	52.0	85.2	44.4	0.340	4
5	107.5	85	54.0	84.5	45.0	0.355	1
6	114.0	92	56.0	83.8	45.6	0.370	2
7	120.5	99	58.0	83.1	46.2	0.385	3
8	127.0	106	60.0	82.4	46.8	0.400	4
9	133.5	113	62.0	81.7	47.4	0.415	1
10	140.0	120	64.0	81.0	48.0	0.430	2
11	146.5	127	66.0	80.3	48.6	0.445	3
12	153.0	134	68.0	79.6	49.2	0.460	4
13	159.5	141	70.0	78.9	49.8	0.475	1
14	166.0	148	72.0	78.2	50.4	0.490	2
15	172.5	155	74.0	77.5	51.0	0.505	3
16	179.0	162	76.0	76.8	51.6	0.520	4
17	185.5	169	78.0	76.1	52.2	0.535	1
18	192.0	176	80.0	75.4	52.8	0.550	2
19	198.5	183	82.0	74.7	53.4	0.565	3
20	205.0	190	84.0	74.0	54.0	0.580	4
21	211.5	197	86.0	73.3	54.6	0.595	1
22	218.0	204	88.0	72.6	55.2	0.610	2
23	224.5	211	90.0	71.9	55.8	0.625	3
24	231.0	218	92.0	71.2	56.4	0.640	4
25	237.5	225	94.0	70.5	57.0	0.655	1
26	244.0	232	96.0	69.8	57.6	0.670	2
27	250.5	239	98.0	69.1	58.2	0.685	3



V	км	мин	кВт*ч	start %	finish %	ВУН/кВт*ч	чел.
28	257.0	246	100.0	68.4	58.8	0.700	4
29	263.5	253	102.0	67.7	59.4	0.715	1
30	270.0	260	104.0	67.0	60.0	0.730	2

6. Этап 4. Проверка программы

6.1. Минимальный набор тестов

Тест	Что должен выявить
T1. Демонстрационный	согласованность программы с приведённым эталоном
T2. Индивидуальный	самостоятельное вычисление ожидаемых значений
T3. 45 минут	ошибку <code>duration_minutes / 60</code> при целочисленном делении
T4. Дробные проценты	потерю дробной части и неверные скобки
T5. Один пассажир	<code>cost_per_passenger</code> должен совпасть с <code>trip_cost</code>
T6. Контрастный	малый расход при большой дистанции или короткая поездка; разумность единиц

Для T2 и ещё одного выбранного теста ожидаемые значения нужно получить независимо от программы: вручную или на обычном калькуляторе. В README.md приведите промежуточные вычисления.

6.2. Таблица тестирования в README.md

ID	Входы	Ожидание	Факт	Итог / объяснение
T1	...	...	...	PASS или причина
T2	...	...	...	PASS или причина
...	...	...	...	...

Округление. Сравнивайте не только уже округлённые строки. Если результаты различаются в третьем знаке, временно выведите 8-10 знаков и проверьте промежуточные величины.

7. Этап 5. Обязательная работа с AI

AI используется как рецензент плана тестирования, а не как источник готовой программы. Отправьте AI свою таблицу входов/выходов, формулы и список T1-T6 без исходного кода.

**Рекомендуемый промпт**

Я выполняю первую лабораторную по С. Программа линейная: без if, циклов, массивов и функций кроме main.

Она рассчитывает расход энергии, среднюю скорость, расход на 100 км, стоимость поездки и стоимость на пассажира.

Ниже мои формулы и тесты: <вставьте формулы и T1-T6>.

Предложи ровно 3 дополнительных теста, каждый должен обнаруживать отдельный риск.

Для каждого укажи конкретные входы, ожидаемое свойство результата и тип ошибки, которую тест способен выявить.

Не пиши и не исправляй код программы.

Выберите не менее двух предложений AI и для каждого примите одно решение: принято, отклонено или изменено. Решение без доказательства не засчитывается.

Предложение AI	Решение	Как проверено	Вывод
Краткое содержание	принято / отклонено / изменено	ручной расчёт / контрпример / документация / запуск	почему это обоснованно

Академическая добросовестность. Сохраните фактический промпт и ответ. Если AI помогал с кодом, это нужно указать отдельно; на защите вы обязаны объяснить и локально изменить любой фрагмент. Непонимание собственного решения ограничивает оценку независимо от запуска программы.

8. Уровневые задания на 7-10 баллов

Правило. Уровни выполняются накопительно: для 8 нужны базовая часть, уровень 7 и уровень 8; для 10 - все четыре расширения. Выполнение расширений не компенсирует неработающую базовую программу.

Уровень 7. Аудит правдоподобного AI-кода

Попросите AI провести ревью приведённого фрагмента. До отправки самостоятельно отметьте подозрительные строки. Затем сравните свой анализ с AI и найдите все ошибки, влияющие на корректность или безопасность выполнения.

**Фрагмент для аудита - не копировать в основное решение**

```
#include <stdio.h>

int main(void) {
    double distance_km, capacity, start_pct, finish_pct, tariff;
    int duration_minutes, passengers;

    scanf("%d", &distance_km);
    scanf("%d %lf %lf %lf %lf %d",
          &duration_minutes, &capacity, &start_pct,
          &finish_pct, &tariff, &passengers);

    double time_hours = duration_minutes / 60;
    double energy_used = capacity * start_pct - finish_pct / 100.0;
    double average_speed = distance_km / time_hours;
    double consumption = energy_used / 100.0 * distance_km;
    double trip_cost = energy_used * tariff;

    printf("Average speed: %d\n", average_speed);
    printf("Consumption: %.2f\n", consumption);
    printf("Trip cost: %.2f\n", trip_cost);
    return 0;
}
```

- ☐ Составьте таблицу: строка - симптом - причина - тест/предупреждение - исправление.
- ☐ Отдельно перечислите ошибки, найденные вами до AI, найденные AI и пропущенные AI.
- ☐ Исправьте фрагмент и подтвердите сборку обязательной командой.

Уровень 8. Что умеет и чего не умеет компилятор

- ☐ Соберите неисправный фрагмент с обязательными флагами и классифицируйте все диагностические сообщения.
- ☐ Исправьте только ошибки типов/форматов, которые обнаружил компилятор, но пока не меняйте формулы.
- ☐ Покажите минимум две оставшиеся логические ошибки на конкретных входах, включая `duration_minutes = 45`.
- ☐ Сделайте вывод: почему чистая сборка необходима, но не доказывает корректность алгоритма.

Уровень 9. Эксперимент float против double

Создайте отдельный файл `precision.c`. Вычислите `energy_used_kwh`, `consumption_kwh_per_100km` и `trip_cost_бун` дважды - в `float` и `double` - и выведите результаты с 10 знаками после точки.

Набор	distance	duration	capacity	start	finish	tariff	passengers
Обычный	ваш V	ваш V	ваш V	ваш V	ваш V	ваш V	ваш V
Стресс	0.7	1	99.9	77.7	77.6	0.333333	3

Для каждого результата рассчитайте абсолютную разность между `float` и `double`, объясните хотя бы одно расхождение и обоснуйте выбор `double` для основной программы.



Уровень 10. Самопроверка через инварианты и метаморфные тесты

Не добавляя ветвлений и циклов, вычислите и выведите три невязки с помощью `fabs()` из `math.h`:

Невязки модели

```
fabs(average_speed_kmh * time_hours - distance_km)
fabs(consumption_kwh_per_100km * distance_km / 100.0 - energy_used_kwh)
fabs(cost_per_passenger_byn * (double)passengers - trip_cost_byn)
```

Затем выполните три пары запусков и проверьте отношения между результатами:

- если удвоить `tariff`, то `trip_cost` и `cost_per_passenger` удваиваются, а физические показатели не меняются;
- если удвоить `passengers`, то `cost_per_passenger` уменьшается вдвое, остальные результаты не меняются;
- если `duration_minutes` удвоить при прежней `distance`, средняя скорость уменьшается вдвое, а энергетические показатели не меняются.

В отчёте сформулируйте каждое свойство до запуска, приведите обе строки входов/выходов и объясните допустимую погрешность сравнения вещественных значений.

9. Структура README.md

1. ФИО, группа, вариант, целевая оценка.
2. Постановка задачи: входы, выходы, единицы и ограничения.
3. Формулы и псевдокод.
4. Инструкция сборки и запуска.
5. Таблица тестов T1-T6 и независимые расчёты.
6. AI-протокол и решения по предложениям AI.
7. Уровневые задания, если выполнялись.
8. Краткий вывод: одна обнаруженная ошибка, один полезный тест, одно ограничение текущего решения.

10. Как формируется оценка

Балл	Что должно быть подтверждено
4-5	Рабочее окружение и компилируемая программа; обязательная часть выполнена частично или содержит локальные недочёты.
6	Полная базовая часть: корректная программа, чистая сборка, README.md, T1-T6, независимые расчёты, AI-протокол и защита.
7	Базовая часть + полностью выполнен уровень 7.
8	Выполнены базовая часть и уровни 7-8.
9	Выполнены базовая часть и уровни 7-9.



Балл	Что должно быть подтверждено
10	Выполнены все уровни 7-10; студент самостоятельно защищает решение и объясняет допустимую погрешность.

Ограничение. Если программа не работает, нет доказательных тестов или студент не понимает собственный код, дополнительные задания не повышают итоговую оценку.

11. Подготовка к защите

- ☐ Покажите чистую сборку из терминала и запуск на назначенном преподавателем наборе.
- ☐ Объясните путь одного значения от ввода до итоговой строки.
- ☐ Поясните, почему $\text{duration_minutes} / 60.0$ не равно $\text{duration_minutes} / 60$.
- ☐ Назовите спецификаторы для `int` и `double` в `scanf()` и `printf()`.
- ☐ Покажите тест, который обнаружит неверные скобки в формуле энергии.
- ☐ Расскажите, какое предложение AI вы отклонили или изменили и каким доказательством руководствовались.
- ☐ Будьте готовы изменить один вход, формат вывода или промежуточную формулу и предсказать результат до запуска.

12. Официальные материалы

MSYS2: <https://www.msys2.org/>

Visual Studio Code - Using GCC with MinGW: [официальная инструкция](#)

GCC - Warning Options: [официальная документация](#)

C language reference: [cppreference \(C\)](#)